

# ARCHITECTURE DESIGN REPORT

AI-Powered Smart Municipal Asset Management and Maintenance Platform

|              |                                                                               |
|--------------|-------------------------------------------------------------------------------|
| Course Code  | CSA10                                                                         |
| Course Title | Software Engineering                                                          |
| Assessment   | CO2 Assessment Tool 2 – Architecture Design Assignment                        |
| CO Assessed  | CO2 – Requirements analysis, scalable architecture and feature prioritization |
| Total Marks  | 25                                                                            |
| Student Name | Jaswanth                                                                      |
| Register No. | 192525214                                                                     |
| Date         | _____                                                                         |

**Prepared from the assigned problem statement and the architecture-design requirements in the supplied assessment document.**

The report covers requirements, architecture selection, architecture diagram, component interactions, quality attributes, and architectural justification.

# 1. System Requirements Analysis

**Problem context.** Municipal corporations manage public assets such as roads, parks, streetlights, drainage systems, public buildings, and water infrastructure. The assigned problem calls for an AI-powered platform that digitizes asset records, monitors asset condition using IoT, predicts maintenance needs, prioritizes repairs, integrates GIS mapping, and generates maintenance analytics.

**System objectives.** The architecture must support continuous asset monitoring, predictive maintenance, prioritized repair planning, centralized infrastructure information, analytics, and improved public service delivery while reducing maintenance costs.

## 1.1 Functional Requirements

- **Asset registration and digital inventory:** create, update, search, and maintain records for municipal assets.
- **Asset condition monitoring:** receive and store condition readings from IoT sensors and inspections.
- **Predictive maintenance:** analyze historical and current asset data to estimate maintenance requirements.
- **Repair prioritization:** rank maintenance tasks using condition, urgency, asset importance, and available work information.
- **GIS-based asset mapping:** display asset locations and related condition or maintenance information on a map.
- **Maintenance scheduling:** create and track repair/maintenance schedules and work status.
- **Analytics and reporting:** generate maintenance, asset-performance, cost, and service reports.
- **Notifications:** alert responsible personnel about high-risk assets, predicted failures, overdue tasks, or important events.
- **User and access management:** provide controlled access for authorized municipal users.

## 1.2 Non-Functional Requirements

| Quality Attribute | Architectural Requirement                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------|
| Scalability       | Support increasing numbers of assets, IoT devices, sensor events, users, and reports.                       |
| Performance       | Process frequent sensor events and provide responsive dashboards, maps, searches, and maintenance views.    |
| Reliability       | Avoid loss of asset and sensor information and provide dependable processing of maintenance events.         |
| Availability      | Keep core monitoring, asset information, and maintenance functions accessible during normal operations.     |
| Security          | Authenticate users, authorize operations by role, protect municipal data, and secure communication.         |
| Maintainability   | Separate responsibilities into manageable components so features can be modified or extended independently. |
| Data integrity    | Maintain consistent asset identifiers, maintenance records, sensor observations, and status information.    |
| Interoperability  | Provide APIs/integration points for GIS, IoT devices, notifications, and future municipal systems.          |

## 1.3 Key Quality Attributes

The most influential attributes are **scalability, maintainability, reliability, availability, performance, security, and interoperability**. Continuous IoT monitoring strongly influences scalability and performance, while municipal operations make reliability and availability important.

## 2. Selection of Suitable Software Architecture

### Selected architecture: Hybrid Architecture (Layered + Event-Driven).

A hybrid architecture is appropriate because the platform combines conventional business functions—asset records, maintenance scheduling, users, dashboards and reports—with continuous IoT event ingestion and AI-based predictive analysis. Layered services organize business functions, while event-driven processing handles sensor events asynchronously.

### 2.1 Architectural Layers and Major Services

- **Presentation layer:** web/mobile dashboards for administrators, engineers and maintenance personnel.
- **API and security layer:** API gateway, authentication, authorization, request routing and validation.
- **Application/service layer:** asset management, maintenance management, scheduling, GIS integration, analytics/reporting, notification and user services.
- **Event-driven IoT layer:** IoT gateway, message broker/event bus, event processing and condition-monitoring services.
- **AI/analytics layer:** predictive maintenance engine, risk scoring, repair-priority logic and analytics processing.
- **Data layer:** relational asset/maintenance database, time-series sensor store, GIS/geospatial store, and reporting/analytics store.

### 2.2 Why This Architecture Fits the Problem

- IoT sensor events can be processed asynchronously without forcing dashboard requests to wait for sensor processing.
- Business functions remain separated into clear services, improving maintainability and future extension.
- AI prediction can be deployed and updated independently from asset-record and user-management functions.
- GIS can be integrated through a dedicated service/API rather than embedding mapping logic throughout the application.
- Different storage approaches can be used for structured asset data, sensor time-series data, and geospatial information.
- Individual processing components can be scaled according to workload.

### 2.3 Architectural Trade-Offs

| Decision                  | Benefit                                                     | Trade-Off                                                |
|---------------------------|-------------------------------------------------------------|----------------------------------------------------------|
| Hybrid architecture       | Matches business workflows and continuous event processing. | More components to deploy and monitor.                   |
| Event-driven IoT          | Handles bursts of sensor events asynchronously.             | Requires message management and monitoring.              |
| Separate AI service       | Allows models to evolve independently.                      | Adds service-to-service and model-management complexity. |
| Dedicated GIS integration | Keeps spatial functionality modular.                        | External GIS availability/integration must be managed.   |

### 3. Software Architecture Diagram

High-level architecture showing users, external systems, secure APIs, business services, event-driven IoT processing, AI analytics, and data stores.

|                                                                                                              |                                                                                         |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>USERS</b><br>Municipal Admin<br>Engineers<br>Maintenance Teams                                            | <b>EXTERNAL / EDGE</b><br>IoT Sensors & Devices<br>GIS Platform<br>Notification Service |
| <b>PRESENTATION LAYER</b><br>Web Dashboard / Mobile Interface                                                | <b>IoT GATEWAY</b><br>Device connectivity & validation                                  |
| <b>API GATEWAY + SECURITY</b><br>Authentication • Authorization • Routing • Validation                       | <b>EVENT BUS / MESSAGE BROKER</b><br>Asynchronous sensor events                         |
| <b>APPLICATION SERVICES</b><br>Asset • Maintenance • Scheduling • GIS • Reporting •<br>Notifications • Users | <b>EVENT PROCESSING</b><br>Condition Monitoring • Rules • Alerts                        |
| <b>AI &amp; ANALYTICS</b><br>Predictive Maintenance • Risk Scoring • Repair<br>Prioritization • Analytics    | <b>INTEGRATION SERVICES</b><br>GIS API • Notification API • Future Municipal APIs       |
| <b>DATA LAYER</b><br>Asset/Maintenance DB • Time-Series Store • GIS Store<br>• Analytics Store               | <b>AUDIT &amp; MONITORING</b><br>Logs • Metrics • System Events                         |

**Communication mechanisms:** HTTPS/REST APIs for synchronous application interactions; message-broker events for IoT and asynchronous processing; secure APIs for GIS and notifications; database connections for persistent storage.

**Logical flow:** Users access dashboards through the API gateway. IoT devices send readings through the IoT gateway to the event bus. Event-processing services evaluate readings and store observations. AI services use asset history and current condition data to estimate maintenance needs and generate risk/priority information. Application services expose this information to maintenance personnel and support scheduling, reporting and notifications.

## 4. Components and Their Interactions

| Component                 | Purpose / Responsibility                                     | Main Interactions                                 |
|---------------------------|--------------------------------------------------------------|---------------------------------------------------|
| Web/Mobile UI             | Dashboards, asset views, maps, schedules and reports.        | Calls API Gateway over HTTPS.                     |
| API Gateway + Security    | Authentication, authorization, routing and validation.       | UI ↔ services.                                    |
| Asset Management          | Maintains digital inventory and asset metadata.              | Asset DB; maintenance; GIS; reporting.            |
| Maintenance Service       | Maintains work orders, records and task status.              | Asset DB; scheduling; notifications; AI priority. |
| Scheduling Service        | Creates and tracks repair schedules and work plans.          | Maintenance; priority engine; notifications.      |
| IoT Gateway               | Receives sensor/device data and validates messages.          | IoT devices ↔ event bus.                          |
| Event Bus                 | Buffers and distributes sensor events asynchronously.        | IoT gateway → event processing.                   |
| Condition Monitoring      | Processes readings and evaluates asset condition.            | Event bus; time-series store; alerts.             |
| AI Predictive Maintenance | Predicts maintenance needs and risk.                         | Data stores; prioritization.                      |
| GIS Integration           | Connects asset records with spatial locations.               | Asset service ↔ GIS platform.                     |
| Reporting & Analytics     | Produces maintenance and infrastructure reports.             | Analytics store; dashboards.                      |
| Notification Service      | Sends alerts and maintenance notifications.                  | AI/rules/scheduling → channels.                   |
| Data Stores               | Persist structured, time-series, spatial and reporting data. | Authorized services read/write.                   |

## 4.1 Overall System Workflow

1. A municipal user signs in and opens the dashboard.
2. The API gateway authenticates the user and routes requests to the appropriate application service.
3. Asset information, location data, maintenance history and current condition information are retrieved from the relevant data stores.
4. IoT sensors continuously send readings to the IoT gateway, which validates and forwards events to the message broker.
5. Condition-monitoring services process events, store observations and identify important conditions.
6. The predictive-maintenance service analyzes available asset and condition data and produces maintenance-risk information.
7. Repair-prioritization logic combines risk/condition information with maintenance records to support scheduling.
8. Maintenance personnel view prioritized tasks, update work status, and record completed maintenance.
9. Reporting and analytics summarize asset condition, maintenance activity and infrastructure performance.
10. Notification services deliver relevant alerts to authorized personnel.

## 4.2 Example Interaction Sequence

**Sensor event** → **prediction** → **maintenance action**: sensor reading → IoT gateway → event bus → condition monitoring → sensor storage → AI analysis → risk/maintenance recommendation → repair prioritization → maintenance dashboard → optional notification.

## 5. Quality Attribute Analysis

| Attribute       | How the Architecture Addresses It                                                                                                                                                            |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability     | Event-driven ingestion scales independently. Stateless API/application services can be replicated. AI processing can scale separately from user-facing services.                             |
| Security        | Centralized authentication and authorization, role-based access, encrypted communication, validation, controlled service access and audit logging.                                           |
| Performance     | Asynchronous sensor processing prevents high-volume IoT traffic from blocking user requests. Operational and analytics workloads can be separated.                                           |
| Reliability     | Message buffering absorbs temporary processing slowdowns. Persistent storage protects asset and maintenance records. Components are isolated by responsibility.                              |
| Maintainability | Clear separation of services and layers limits the impact of changes. AI, GIS, notification and reporting can evolve independently.                                                          |
| Availability    | Core services can be deployed redundantly where required. Event processing can continue asynchronously when a downstream consumer is temporarily unavailable, subject to operational design. |

### 5.1 Quality Attribute Priorities

- **Highest priority:** reliability, availability, security and scalability because the platform supports public infrastructure and continuous monitoring.
- **Next priority:** performance and maintainability because users need timely information and the system must support long-term municipal operations.
- **Supporting priority:** interoperability so GIS, IoT and future municipal systems can be integrated through stable interfaces.

## 6. Architectural Justification and Future Scope

The proposed hybrid architecture directly reflects the assigned problem. The system is not only a conventional record-management application: it must receive continuous IoT data, perform predictive analysis, integrate GIS information, prioritize repairs, and produce operational reports. Combining layered organization with event-driven processing therefore provides a balanced architecture.

### 6.1 Architectural Decisions

- 1. Use a secure API gateway:** provides a controlled entry point for dashboards and future clients while centralizing authentication, authorization and routing.
- 2. Separate business services:** asset, maintenance, scheduling, reporting and notification responsibilities remain modular.
- 3. Use event-driven IoT processing:** sensor events are handled asynchronously so high event volume does not directly overload interactive functions.
- 4. Separate AI processing:** predictive maintenance and risk scoring can improve independently as analytical requirements evolve.
- 5. Use purpose-oriented data stores:** structured operational, time-series, geospatial and reporting data have different access patterns.
- 6. Use modular integrations:** GIS and notification capabilities connect through dedicated services/APIs, supporting future replacements or additions.

### 6.2 Future Enhancements

- Add more sensor types and additional categories of municipal assets.
- Introduce more advanced predictive models as sufficient historical data becomes available.
- Add mobile workflows for field inspections and maintenance updates.
- Integrate additional municipal systems through APIs.
- Extend analytics with asset lifecycle, maintenance-cost and infrastructure-utilization dashboards.
- Add stronger operational monitoring, automated backup/recovery and high-availability deployment where required.

### 6.3 Final Architectural Justification

The architecture supports the objectives of digitizing municipal asset records, continuously monitoring conditions, predicting maintenance requirements, prioritizing repairs, integrating GIS mapping, generating maintenance analytics, improving infrastructure utilization, and enhancing public service delivery. It also provides a structured foundation for reducing maintenance costs and supporting data-driven infrastructure management.

## 7. Expected Outcomes and Conclusion

The assigned problem statement identifies the following expected outcomes: improved infrastructure reliability, reduced maintenance costs, better public asset utilization, faster maintenance planning, enhanced municipal services, data-driven infrastructure management, increased operational efficiency, and improved citizen satisfaction.

The proposed architecture supports these outcomes through a centralized digital asset inventory, continuous IoT monitoring, predictive maintenance analysis, repair prioritization, GIS integration, analytics and notifications. The hybrid architecture provides a scalable and maintainable foundation for a smart municipal asset management platform.

### Assessment Alignment

| Assignment Requirement                       | Covered in This Report                                                                        |
|----------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1. Analyze System Requirements               | Section 1 – functional, non-functional and quality requirements                               |
| 2. Select a Suitable Software Architecture   | Section 2 – Hybrid (Layered + Event-Driven) architecture and justification                    |
| 3. Draw the Software Architecture Diagram    | Section 3 – high-level architecture diagram                                                   |
| 4. Explain Components and Their Interactions | Section 4 – component responsibilities and workflow                                           |
| 5. Discuss Quality Attributes                | Section 5 – scalability, security, performance, reliability, maintainability and availability |
| 6. Justify Architectural Decisions           | Section 6 – decisions, trade-offs and future enhancements                                     |

**Student: Jaswanth | Register No.: 192525214**